Rapport de projet

Base de réseaux

MCHEIK Mohamed
KENNES Guillaume

Tables des matières

Contents

1. Introduction 3
2. Architecture du protocole 3
2.1 Format des paquets 3
2.2 Séquences de contrôle 3
3. Implémentation 4
3.1 Structure des classes 4
3.2 Détails d'implémentation 4
4. Fonctionnalités principales et forces du projet 5
4.1 Fiabilité 5
4.2 Contrôle de flux 5
4.3 Gestion de la connexion 5
4.4 Compatibilité 5
5. Faiblesses du projet 5
5.1 Faiblesses 5
6. Conclusions 6

1. Introduction

Ce projet consiste à implémenter un protocole fiable de transfert de fichiers au-dessus d'UDP, en s'inspirant des grands principes de TCP. L'idée de départ est simple : UDP est rapide mais ne garantit rien ni l'ordre, ni la livraison. L'objectif était donc de construire, à la main, les mécanismes qui rendent le transfert de données digne de confiance.

Pour y arriver, plusieurs mécanismes ont été mis en place :

- Numéros de séquence pour maintenir l'ordre des paquets
- Accusés de réception (ACK) cumulatifs
- Fenêtre glissante pour le contrôle de flux
- Détection et la retransmission rapide des paquets perdus
- Séquences de handshake et de fermeture de connexion

2. Architecture du protocole

2.1 Format des paquets

Chaque paquet est composé d'un header fixe de 5 bytes suivi des données de taille variable. Voici la structure du header :

- Bytes 0–1 : dataSize (16 bits, big-endian) taille des données en octets
- Bytes 2–3 : sequenceNumber (16 bits, big-endian) numéro de séquence (0 à 65 535)
- Byte 4 : flags (8 bits)
  - Bit 3 : SYN (synchronisation / handshake)
  - Bit 2 : ACK (accusé de réception)
  - Bit 1 : FIN (fermeture de connexion)
  - Bit 0 : RST (reset / erreur)
- Byte 5+ : data, données utiles, jusqu'à 1 019 octets

2.2 Séquences de contrôle

Le protocole distingue quatre types d'échanges entre l'émetteur et le récepteur.

Handshake (3-way) avant tout transfert, les deux parties s'échangent leurs numéros de séquence initiaux :

- Sender → Receiver : SYN (seq=rand)
- Receiver → Sender : SYN-ACK (seq=rand)
- Sender → Receiver : SYN (seq=seq+1)

Transfert de données : l'émetteur envoie des paquets DATA, le récepteur répond par des ACK cumulatifs. La fenêtre glissante est de 256 paquets, et une retransmission rapide est déclenchée après 3 ACK dupliqués.

Fermeture (3-way) : la connexion se ferme proprement et de façon symétrique :

- Sender → Receiver : FIN
- Receiver → Sender : FIN+ACK
- Sender → Receiver : ACK final

Reset (RST) : en cas d'erreur, un signal RST interrompt immédiatement la connexion. Un message d'erreur est affiché sur stderr, et les données déjà reçues sont conservées.

3. Implémentation

3.1 Structure des classes

Le projet est organisé autour de quatre fichiers Java principaux.

Object.java regroupe les structures de données fondamentales : la classe Packet, qui sait se sérialiser (toBytes()) et se désérialiser (fromBytes()), ainsi que connectedInfo, qui stocke les informations de connexion socket, adresse, port, et numéros de séquence partagés.

Utils.java centralise les fonctions utilitaires : envoi et réception de paquets UDP, construction des ACK, gestion du wraparound 16-bit, et formatage des logs.

Sender.java prend en charge la connexion (3-way handshake avec tentatives), le transfert du fichier avec gestion de la fenêtre et retransmission, et enfin la fermeture propre.

Receiver.java s'occupe côté récepteur : établissement de la connexion, réassemblage des paquets hors ordre (via TreeMap), envoi des ACK cumulatifs.

3.2 Détails d'implémentation

Côté Sender :

- 3 tentatives SYN avec timeout avant d'abandonner
- Buffer de retransmission géré avec une TreeMap
- La fenêtre avance uniquement jusqu'au dernier séquence confirmé de façon contiguë
- La perte est détectée à partir de 3 ACK dupliqués avec des séquences distinctes
- Retransmission rapide : nextIndexNum est remis à base

Côté Receiver :

- TreeMap pour réassembler les paquets arrivés dans le désordre
- ACK cumulatif : confirme le dernier seq continu reçu
- Le base n'avance que si le prochain paquet attendu est bien là
- En cas de RST : message sur stderr et conservation des données déjà reçues

4. Fonctionnalités principales et forces du projet

4.1 Fiabilité

- Numéros de séquence pour garantir l'ordre des paquets
- ACK cumulatifs pour confirmer la réception
- Retransmission automatique des paquets non confirmés
- Détection de pertes par ACK dupliqués
- Timeout pour détecter les connexions inactives

4.2 Contrôle de flux

- Fenêtre glissante de 256 paquets
- Limitation du nombre de paquets simultanément en transit
- Avance progressive et ordonnée de la fenêtre

4.3 Gestion de la connexion

- 3-way handshake (SYN / SYN-ACK / SYN)
- 3-way close (FIN / FIN+ACK / ACK)
- Tentatives de reconnexion automatiques
- Signalisation d'erreur par RST

4.4 Compatibilité

- Fonctionne avec Java 17 et versions supérieures
- Compatible localhost et réseau distant
- Port configurable
- Testé sur des fichiers allant jusqu'à 200 Ko

5. Faiblesses du projet

5.1 Faiblesses

- Pas de contrôle de congestion. La fenêtre est fixe, sans adaptation aux conditions réseau (contrairement à TCP réel).
- Échelle limitée. Numérotation 16 bits seulement : wraparound rapide sur gros transferts/flux longs.
- Concurrence limitée. recepteur plutôt mono-session (pas de gestion multi-clients simultanés).
- Paramètres codés en dur. Taille fenêtre, timeout, taille des données transférer sont fixes, donc moins flexible selon les environnements.
- Sécurité absente. Pas de chiffrement, pas d’authentification, pas de protection contre injection/replay.

6. Conclusions

Ce projet est une démonstration concrète qu'on peut reconstruire la fiabilité de TCP par-dessus UDP, à condition de bien implémenter les bons mécanismes. La fenêtre glissante, les ACK cumulatifs et la retransmission rapide forment le cœur du système et les tests montrent que l'ensemble tient la route.

Le protocole respecte les spécifications et intègre toutes les fonctionnalités attendues : header compact de 5 octets, handshake et fermeture bien formés, gestion des erreurs (RST, timeouts), fenêtre dynamique, et logs détaillés pour faciliter le débogage.

Points forts de l'implémentation :

- Implémentation correcte et complète
- Code lisible et bien structuré
- Gestion d'erreurs robuste
- Compatibilité Java 17

Des améliorations restent envisageables pour une version future :

- Compression des données
- Chiffrement de bout en bout
- Multiplexing (plusieurs fichiers en parallèle)